

OS Bug Bounty 2026

Consolidated Team Findings Report
National Linux Distribution Bug Bounty | Target: Hardened Debian-based Linux Distro | 36-hour event

This report consolidates **14 findings** submitted by the team across five problem statements (PS-01 Kernel/Process Security, PS-02 Authentication & Privilege Management, PS-03 Supply Chain / Package Integrity, PS-04 Network Security, PS-07 Filesystem & Permissions, PS-08 Logging & Auditing). Each entry below is written in plain language first, followed by the underlying OS mechanism, so it can be understood without prior context. Severity labels reflect the team's assessment based on testing, not the official judging outcome.

Contributor	Findings	Problem Statements Touched
Noorul	5	PS-01, PS-02, PS-03, PS-04, PS-08
Shylee & Noorul	1	PS-03
Jeevan	2	PS-02
Jairus	4	Boot / AV / GRUB (inferred: PS-04, PS-07, PS-08)
Dhanya	2	PS-04, PS-08

PS-01 - Kernel / Process Security

1. Unrestricted /proc/<pid>/status exposure (missing hidepid)

Reported by: Noorul | Problem Statement: PS-01 | **Likely severity: Medium**

What this area covers: Linux exposes live information about every running process through a virtual filesystem at `/proc`. Each process gets its own folder, e.g. `/proc/1234/status`, showing memory usage, UID, security flags, and more. The **hidepid** mount option controls whether ordinary users can see other users' (including root's) process details, or only their own.

In plain terms: On this system, `/proc` is mounted without `hidepid` restrictions. That means any logged-in user, even with no special privileges, can open `/proc/<pid>/status` for root-owned processes and read security-sensitive fields normally meant to be hidden from them.

Why it matters: This is an information-disclosure issue: it doesn't let an attacker take over the system by itself, but it hands them a map of what's running as root, their memory footprints, and security attributes — useful reconnaissance for a follow-up privilege escalation attempt.

PS-02 - Authentication & Privilege Management

2. Broken sudo hardening: unsupported 'Defaults !authenticate_override' directive

Reported by: Noorul (also identified independently by Jeevan) | Problem Statement: PS-02 | **Likely severity: Low**

What this area covers: The file `/etc/sudoers` controls who is allowed to run commands as root using 'sudo', and can also set security defaults for how sudo behaves. One such setting was meant to be 'authenticate_override', intended to change sudo's authentication requirement.

In plain terms: The version of sudo installed on the OS does not actually support this directive. When sudo reads it, it prints a harmless parse warning and simply ignores the line — but authentication (the password prompt) still happens normally afterward. So the control that was 'supposed' to be there quietly does nothing, without any error that would alert an admin.

Why it matters: Confirmed through testing that sudo still asks for a password and evaluates permissions correctly, so this is not an authentication bypass. The real risk is a false sense of security: whatever hardening policy this directive was meant to enforce is silently not applied, and there's no visible warning during normal use to tell an administrator that it failed.

3. Sudoers privilege escalation misconfiguration

Reported by: Jeevan | Problem Statement: PS-02 | **Likely severity: Medium**

What this area covers: Beyond the broken directive above, the sudoers configuration itself was reviewed for rules that grant more access than intended — for example, overly broad 'ALL' rules or NOPASSWD entries that let a user run root-level commands without any authentication step at all.

In plain terms: A misconfigured privilege rule was identified in the sudoers setup that could allow a user to gain root-level command execution beyond what their intended role should permit.

Why it matters: Sudoers misconfigurations are one of the most direct paths to full system compromise on Linux, since they hand out root command execution directly — this class of bug is consistently high-value in any privilege management review.

4. Root account set to `/usr/sbin/nologin` instead of being fully disabled

Reported by: Jeevan | Problem Statement: PS-02 | **Likely severity: Low**

What this area covers: On a hardened Linux system, the root account's login shell is often set to `/usr/sbin/nologin` to stop anyone from getting an interactive shell by logging in directly as root (e.g. via 'su' or a console login prompt). This is different from actually locking or disabling the account's password in `/etc/shadow`.

In plain terms: The OS uses nologin as the shell, but this only blocks the shell from launching after a successful login — it does not stop the authentication attempt itself. That means the system still accepts and checks a password for the root account before it fails, rather than refusing the login attempt outright the way a properly disabled/locked account would.

Why it matters: This distinction matters for brute-force and credential-stuffing resistance: an attacker can still attempt to authenticate as root and get meaningful feedback from the auth process, instead of being shut out immediately by a disabled account.

5. Implicit global trust model in APT repository authentication

Reported by: Noorul | Problem Statement: PS-03 | Likely severity: Medium

What this area covers: APT (the package manager) decides whether to trust a software package by checking it against cryptographic signing keys it has been told to trust, stored under `/etc/apt/trusted.gpg.d/` and similar locations. Once a key is trusted, APT accepts anything signed by it, system-wide, with no further scoping.

In plain terms: The review found that the OS's APT trust model treats any repository signed by an installed key as globally and implicitly trusted, without per-repository restriction. There is no mechanism limiting which packages or repository paths a given trusted key is allowed to sign for.

Why it matters: If any single trusted signing key is ever compromised or a rogue repository is added using a valid key, APT will accept and install packages from it without additional scrutiny — a single point of failure in the software supply chain.

6. Static `/etc/hosts` mapping hardcodes APT repository resolution

Reported by: Shylee & Noorul | Problem Statement: PS-03 | Likely severity: Medium

What this area covers: Normally, when the system needs to reach a package repository (e.g. `repo.example-os.org`), it looks up the domain's IP address via DNS at the time of the request. The vendor's hosts-setup script instead adds a fixed entry to `/etc/hosts`, hardcoding that domain to one specific IP address.

In plain terms: Because the mapping is hardcoded rather than resolved dynamically, the system will always reach the package repository at that one fixed IP, even if the legitimate service has moved. This creates a rigid trust dependency: if that specific server is ever compromised, retired, or its IP reassigned, there is no DNS-level flexibility to redirect to a legitimate replacement, and the system won't notice anything is wrong.

Why it matters: This weakens resilience of the software update chain and makes IP-based hijacking or impersonation of that single address a durable attack path, since the system will trust it indefinitely by design.

PS-04 - Network Security

7. Unauthenticated MITM route hijacking via spoofed ICMP redirects

Reported by: Noorul | Problem Statement: PS-04 | Likely severity: High

What this area covers: ICMP redirect messages are a legitimate part of the IP protocol: a router can tell a host 'send your traffic for this destination via a different, better route instead.' The Linux kernel, by default, trusts and acts on these messages without verifying the sender is actually authorized to issue them.

In plain terms: Because ICMP redirects are accepted without authentication, an attacker on the same local network can forge (spoof) a redirect message and trick the machine into rerouting its traffic through an attacker-controlled path — a classic man-in-the-middle setup — without needing any credentials.

Why it matters: This allows silent interception or manipulation of network traffic from an unauthenticated position on the LAN. It's a kernel-level networking hardening gap (the fix is typically disabling `accept_redirects/secure_redirects` via `sysctl`), not something a user-level fix can address.

8. Firewall service disabled by default

Reported by: Dhanya | Problem Statement: PS-04 | **Likely severity: High**

What this area covers: A firewall inspects and filters incoming/outgoing network traffic based on rules. For it to do anything, its service needs to be both installed and actively running.

In plain terms: The firewall components are present on the system, but the service that enforces the rules is disabled by default and not running, so the system currently has no active network traffic filtering at all — despite appearing, on the surface, to have firewall software installed.

Why it matters: This leaves every network-facing port and service exposed with no filtering layer, directly compounding risks like the ICMP redirect issue above and any exposed listening services.

7b. grub.timer not loaded during boot, silently disabling the firewall auto-start

Reported by: Jairus | Problem Statement: PS-04 (category inferred) (category inferred) | **Likely severity: High**

What this area covers: systemd timers (like grub.timer here) are used to trigger services on a schedule or at boot — in this case, apparently intended to (re-)enable the firewall service periodically or at startup.

In plain terms: During boot, grub.timer is not actually loaded/active, which means the firewall auto-enable step it was supposed to trigger never runs. The system nonetheless behaves as if it is refreshing every 5 seconds — a status message that does not reflect reality.

Why it matters: This combines two problems: the firewall doesn't get enabled at boot as intended, and the system reports a false 'refreshing every 5 seconds' status, meaning an administrator monitoring the system would have no visible indication anything is wrong.

PS-07 - Filesystem & Permissions

9. GRUB plaintext password exposed in a world-readable boot script

Reported by: Jairus | Problem Statement: PS-07 (category inferred) (category inferred) | **Likely severity: High**

What this area covers: GRUB is the bootloader — the program that runs before the OS itself and can be password-protected so that only an authorized person can edit boot parameters or boot into recovery/single-user mode. That password is normally supposed to be stored as a hash, not readable plaintext.

In plain terms: A script involved in the GRUB setup contains the password in plaintext, and the file's permissions allow any local user to read it (world-readable).

Why it matters: Anyone with a local session, or physical/USB access to boot into a recovery environment, can read this file and obtain the GRUB password directly, then use it to alter boot parameters (e.g. boot into a root shell), completely bypassing the intended protection. Physical-access attacks against this machine become close to trivial.

10. GRUB configuration template with password hash left world-readable

Reported by: Jaiirus | Problem Statement: PS-07 (category inferred) (category inferred) | Likely severity: Medium

What this area covers: Separate from the plaintext password script above, the GRUB config template file itself stores a hashed version of the password. Even a hash, if readable by anyone, can potentially be cracked offline, and its presence confirms this file is not being properly excluded/protected during the boot process.

In plain terms: This template file, containing the password hash, is also left world-readable rather than restricted to root. This confirms the file is not excluded from normal user access during boot handling, meaning the intended protection mechanism for the bootloader is not actually being enforced.

Why it matters: Combined with finding 9, this shows a systemic gap in how the OS protects boot-time security controls in a physical-access threat model — a hostile actor with a login session or removable media access has multiple ways to defeat GRUB protection.

PS-08 - Logging, Auditing & Integrity Monitoring

11. Tripwire runs daily via cron against a baseline database that was never created

Reported by: Noorul | Problem Statement: PS-08 | Likely severity: High

What this area covers: Tripwire is a file-integrity monitoring tool: it compares the current state of important system files against a previously saved 'baseline' database, to detect unauthorized changes. That baseline has to be initialized once before Tripwire can do any real comparison.

In plain terms: The baseline database was never initialized — the exact file the check script expects is missing ('file not found'), and the reports folder is empty. Despite this, a cron job runs Tripwire every day with no error message ever surfacing to alert anyone that the check is doing nothing.

Why it matters: This is a false sense of security at its clearest: the system appears to have active file-integrity monitoring in its cron schedule, but it has silently never actually monitored anything since installation. Any tampering with system files during this entire period would have gone completely undetected.

12. Virus scanning service disabled while reporting zero threats found

Reported by: Dhanya | Problem Statement: PS-08 | Likely severity: High

What this area covers: An antivirus/malware scanner needs to actually run to be meaningful. If its service is stopped, it should ideally report 'not running' rather than a clean result.

In plain terms: The scanning service is disabled, yet the system (or its dashboard/status output) reports zero threats found — as if a scan completed successfully and the machine is clean.

Why it matters: This is a false-positive-safety report: an administrator glancing at the status would reasonably conclude the machine is protected and clean, when in reality no scanning has taken place at all. This is arguably worse than having no AV feature at all, because it actively misleads.

12b. GUI/CLI falsely reports clean scans due to missing AV database

Reported by: Jairus | Problem Statement: PS-08 (*category inferred*) (*category inferred*) | **Likely severity: High**

What this area covers: For a virus scanner to check files, it needs an up-to-date signature database of known threats. Without that database, it cannot meaningfully evaluate any file.

In plain terms: The AV database is missing, so during boot and on-demand scans the GUI/CLI reports files as 'scanned' and clean, when in reality no actual scanning against threat signatures occurred — the tool is recording files as checked without ever having the data needed to check them.

Why it matters: Same class of issue as finding 12, from a different angle: it's a second, independent confirmation that The OS's malware-protection status reporting cannot be trusted at face value, which is a serious issue for any deployment that relies on that status for compliance or trust decisions.

Notes for Reviewer

- Severity ratings above reflect the team's own assessment after testing, and may differ from official competition scoring.
- Items marked '*category inferred*' were not explicitly tagged with a problem-statement number in the raw submission notes; the mapping shown is the team's best-fit interpretation based on the technical area affected, and should be verified against the official submission portal.
- Several findings across PS-04 and PS-08 (firewall auto-start, AV/Tripwire status reporting) share a common root theme: **security controls that are inactive while the system continues to report a healthy/clean/active status**. This pattern may be worth flagging as a systemic observation in addition to the individual bug reports.